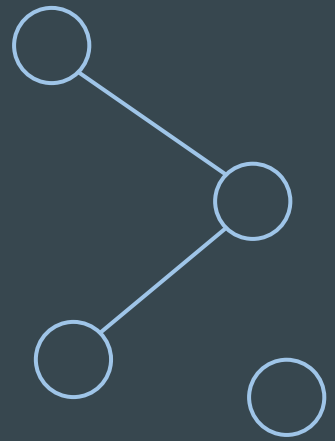
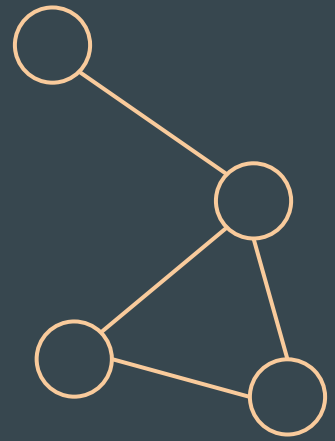
Trees

Binary Search Tree, KD-Tree,
Minimum Spanning Tree, Trie, Segment Tree

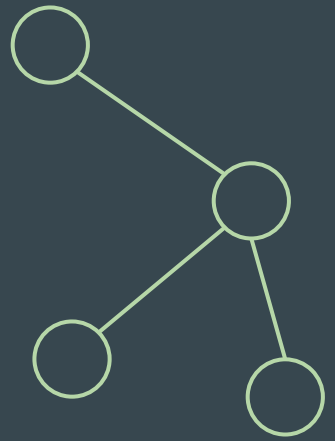
What is a tree ?



acyclic graph



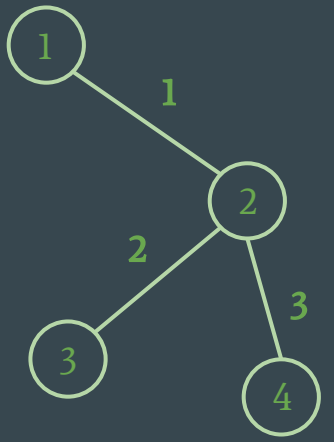
connected graph



tree = connected + acyclic

Some properties of trees...

- A tree has N-1 edges for N vertices
- A tree is a connected & acyclic graph
- Remove any edge from the tree and it disconnects the graph
- In the general case vertice can have any degree
    - Some trees are more specific : binary trees, k-trees

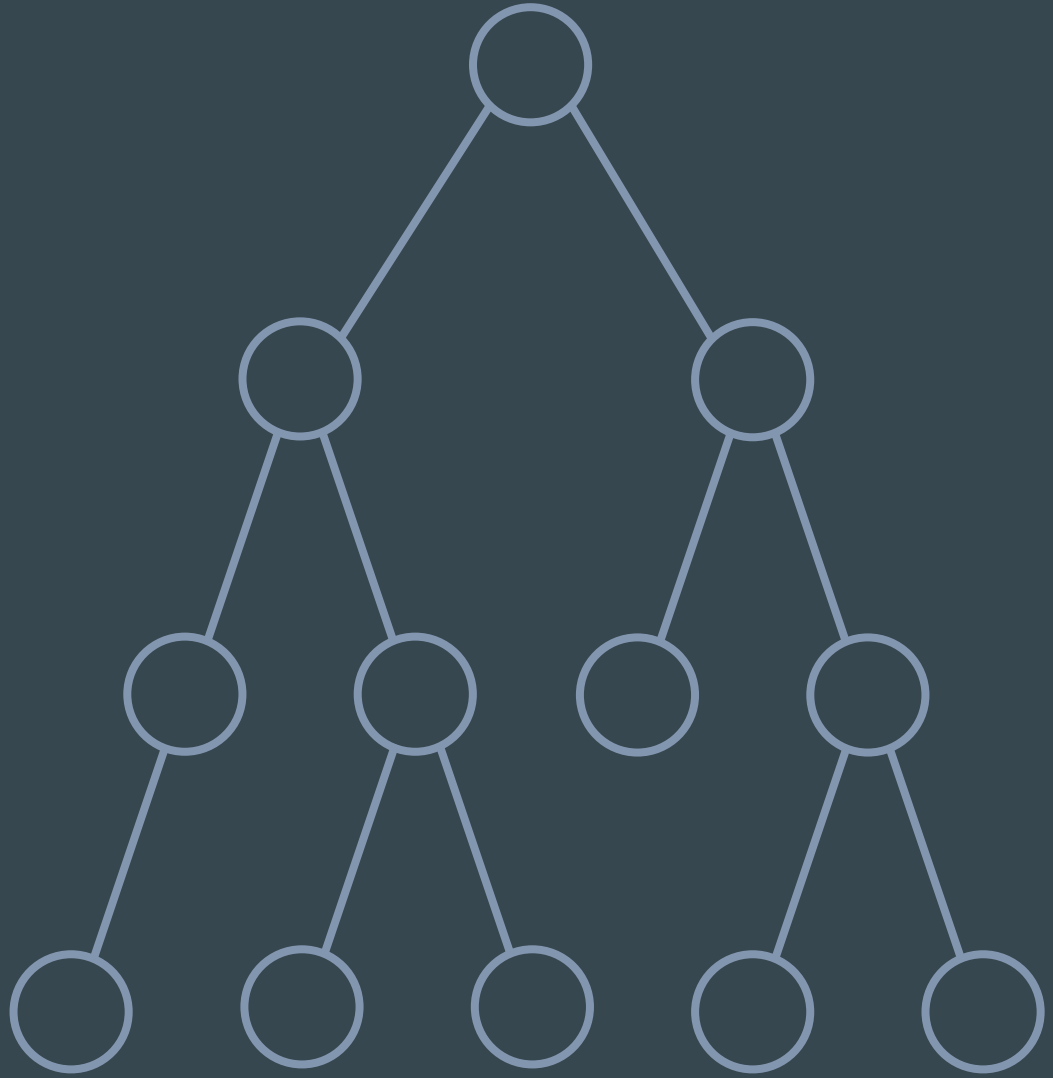


Binary search trees and K-dimensional trees

Binary trees

A tree is a connected graph (no island) with no cycles.

We'll use rooted tree (only one node has no parent), specifically with 2 children (max) on each node.



What is a binary search tree ?

There are only two rules

For each node :

- All the nodes under its left child have lower values
- All the nodes under its right child have higher values

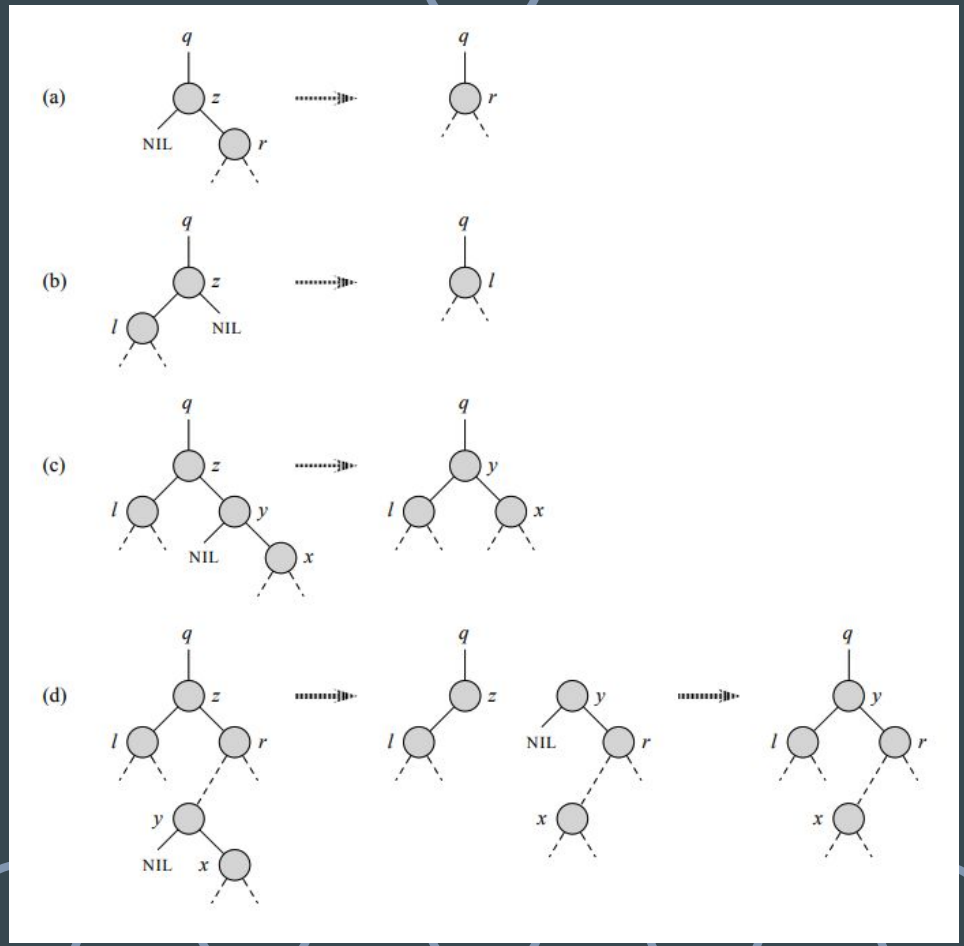
There are multiple types of binary search trees !
(like balanced trees)

What is a binary search tree ?

To remove a node, you can simply remove its value (set it to null), and reinsert all of its children.

But if you do it too often, you'll end up with a **bamboo** !

Here are some algorithms to rebalance your tree after a deletion.



What it's for ?

They're used to perform a search in logarithmic time.
-> `O(log(n))`

For example: searching for the closest point to a given coordinates.

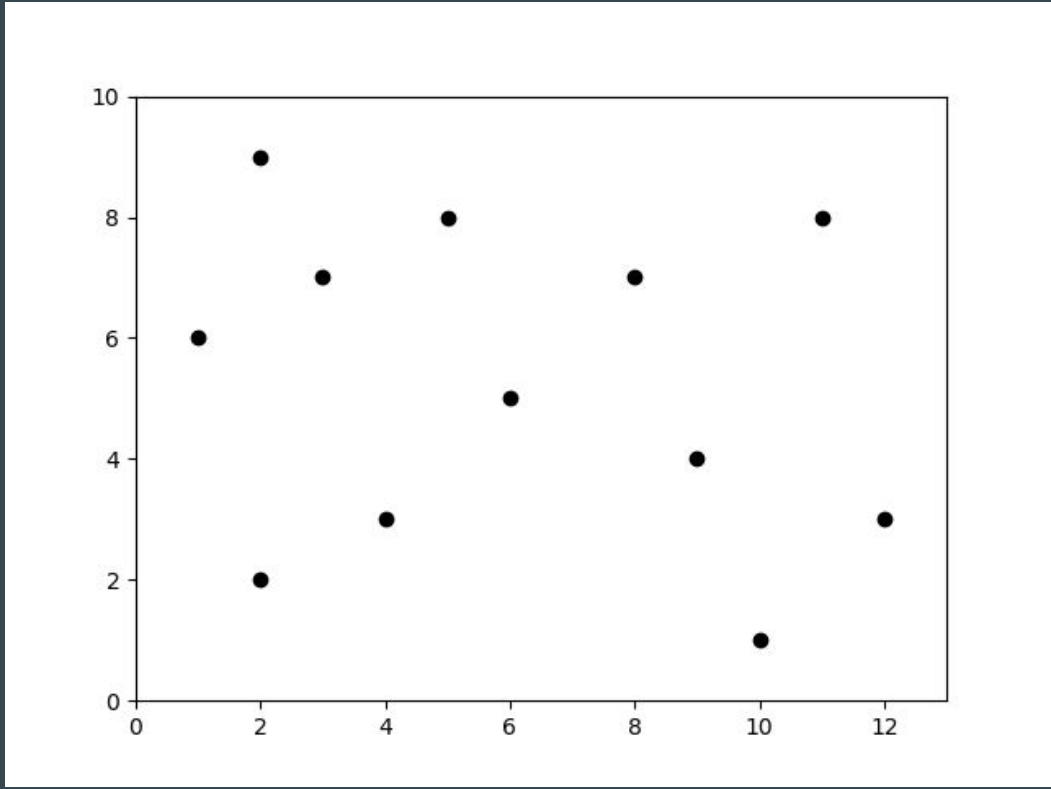
In 1D, the algorithm is trivial.

Multi-dimensionnal BST: KD-trees

That's where it gets complicated: the search algorithm also works for any higher dimension.

The tree is made so that each level divides a dimension.

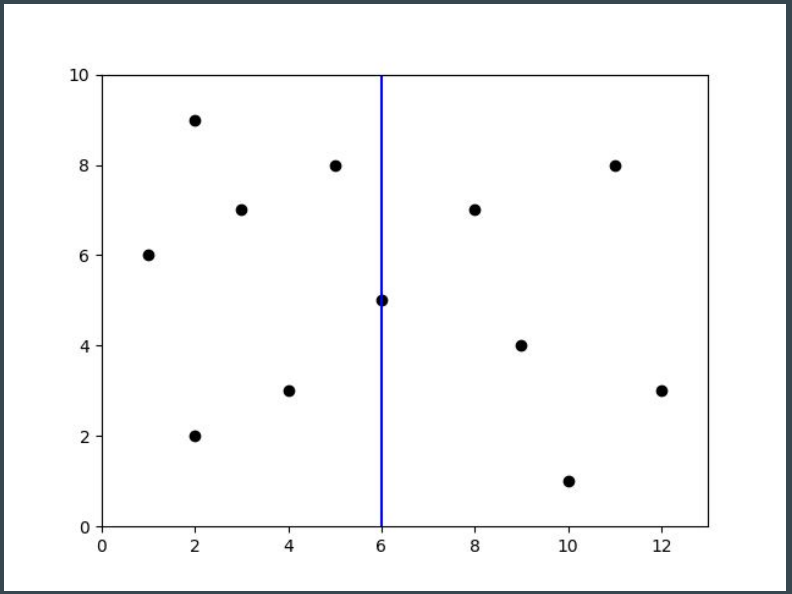
Let's build the tree for this 2D scatter.



Multi-dimensionnal BST: 2D-trees

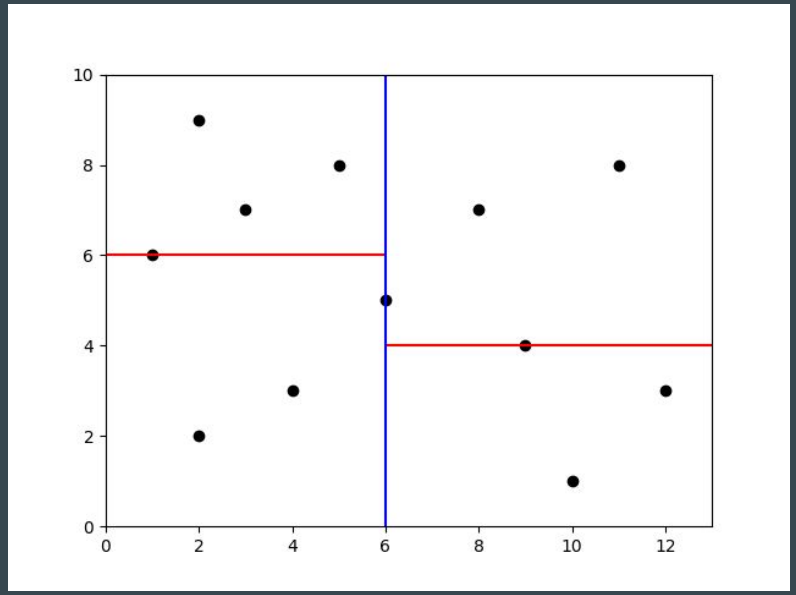
(6, 5)

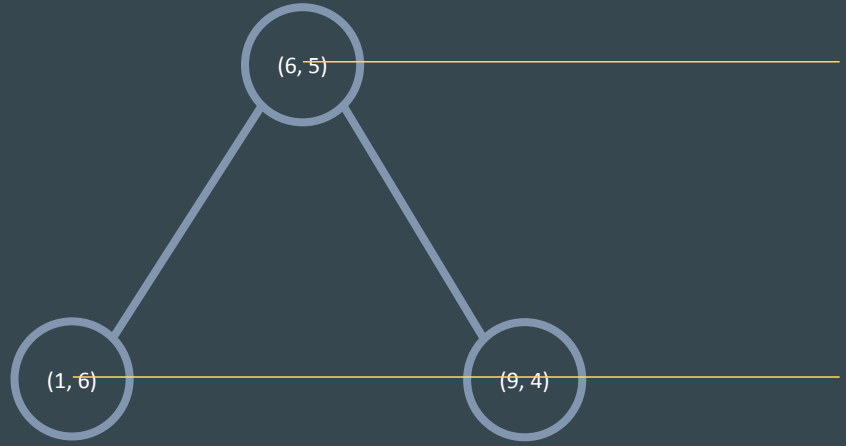
First, we cut the plane in half through the point closest to the middle of the x axis :



Multi-dimensionnal BST: 2D-trees

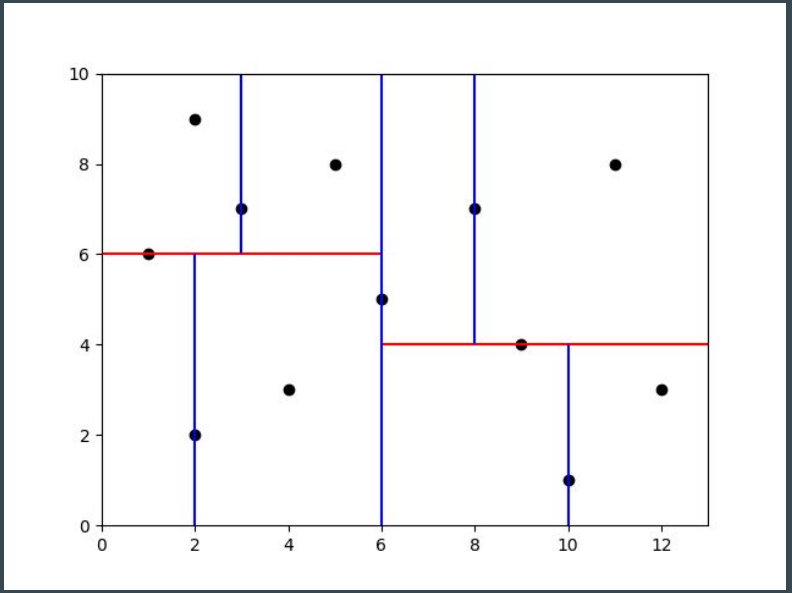
Then we cut the left part and the right part in halves :

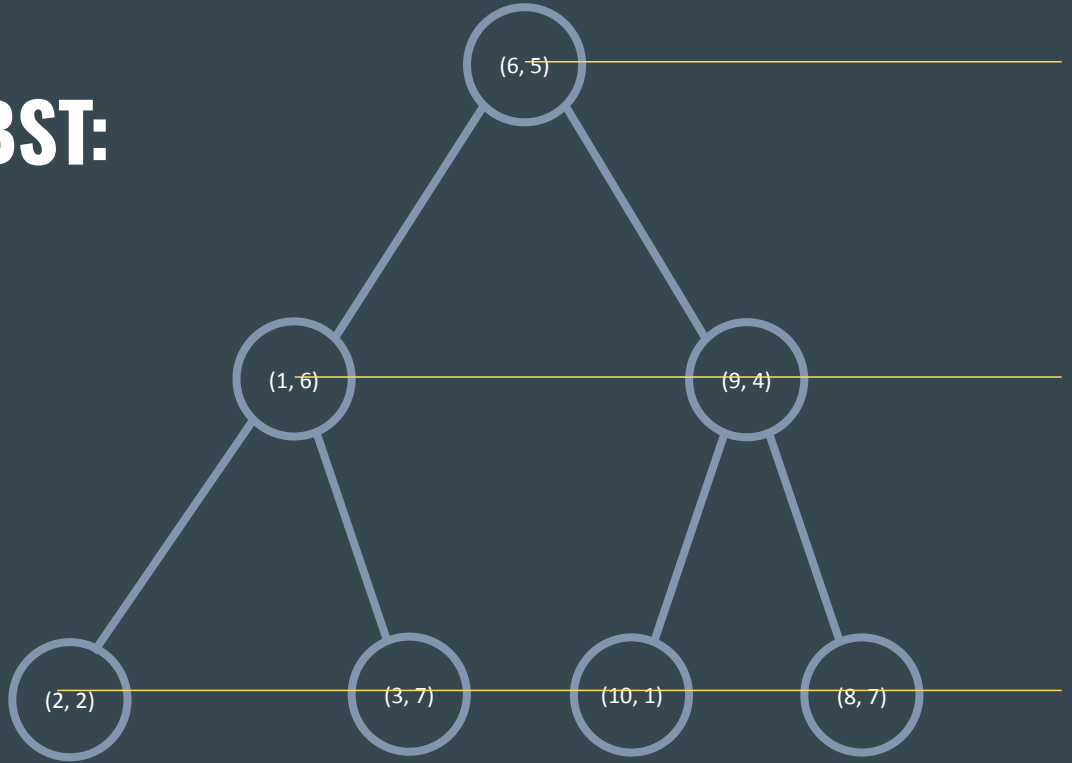




Multi-dimensionnal BST: 2D-trees

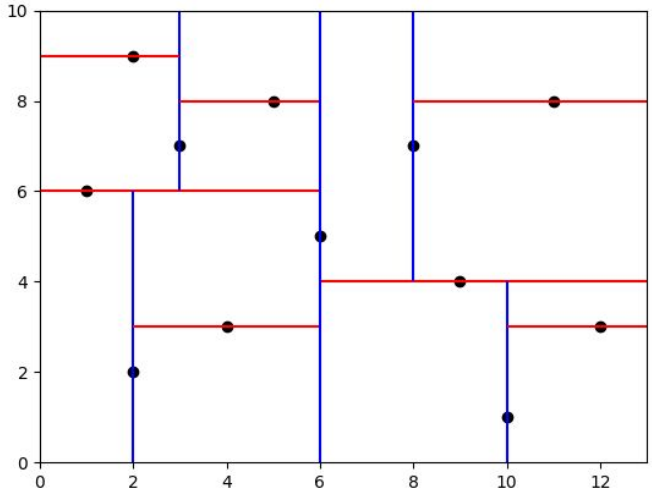
And then repeat until all the points are in the tree :

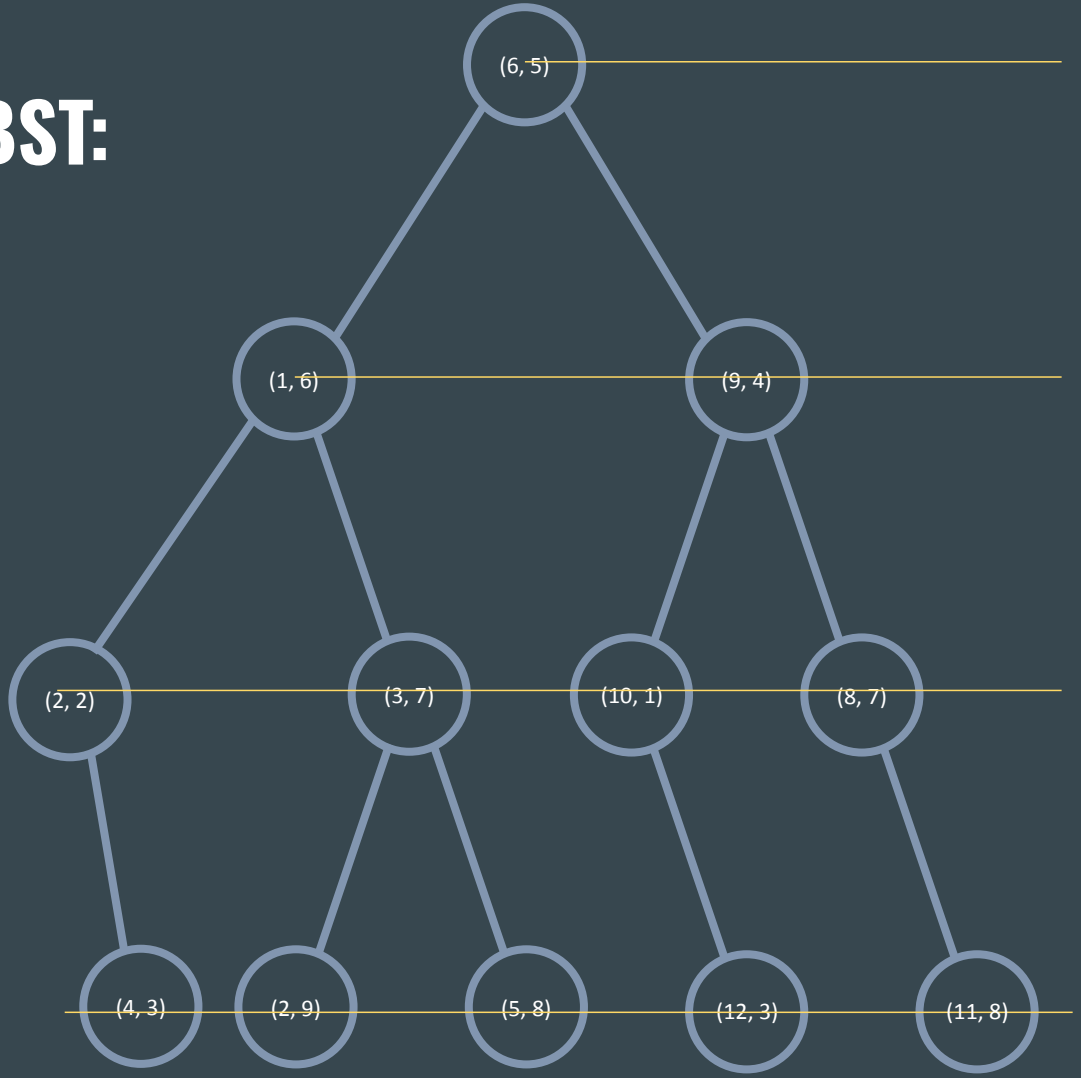




Multi-dimensionnal BST: 2D-trees

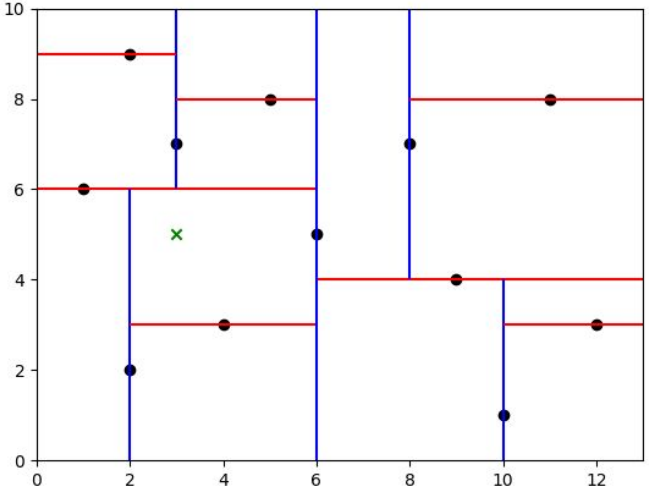
And then repeat until all the points are in the tree :

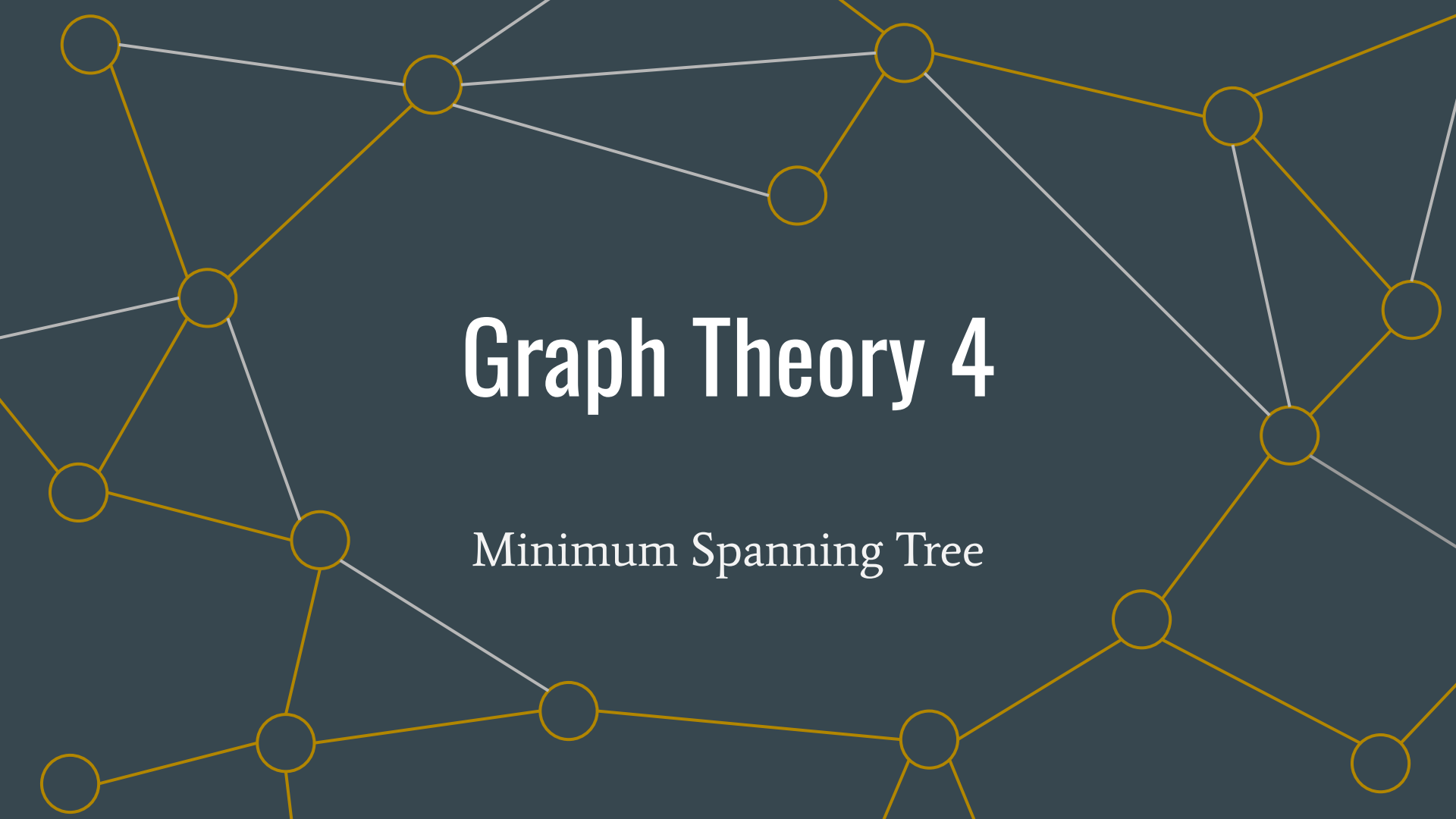




Multi-dimensionnal BST: 2D-trees

Now we can search which point of the 2D scatter is closer to given coordinates in O(log (n))




Graph Theory 4
Minimum Spanning Tree

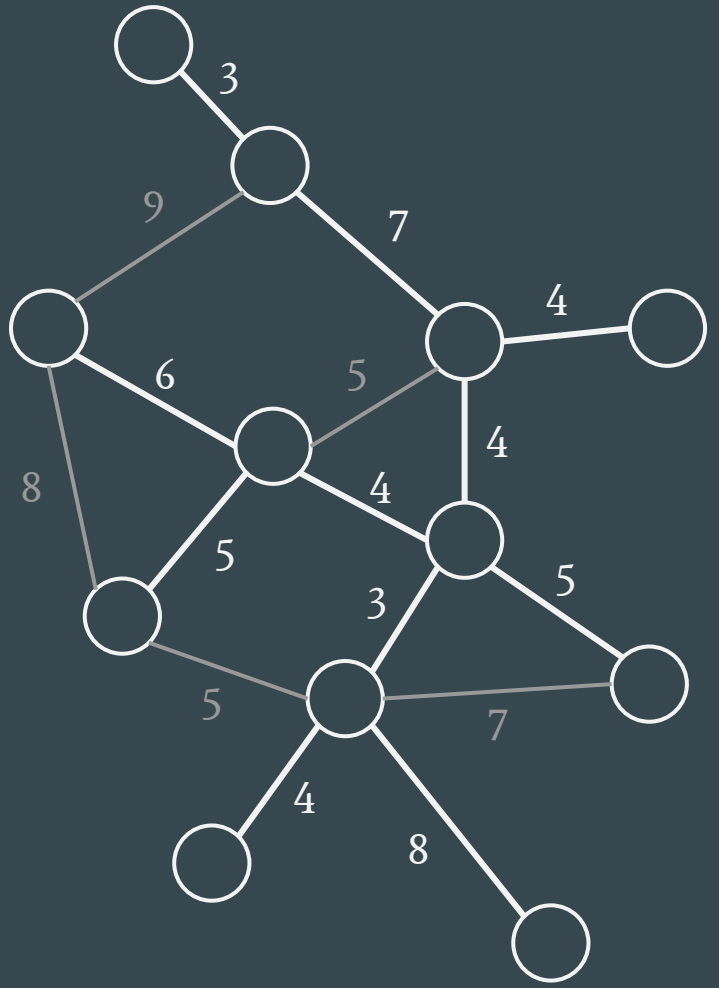
Minimum spanning tree

We are given a connected graph with weighted edges

→ what is the minimum cost to connect all the nodes?

(on this graph: 53)

/!\ Multiple MST are possible



Prim's algorithm

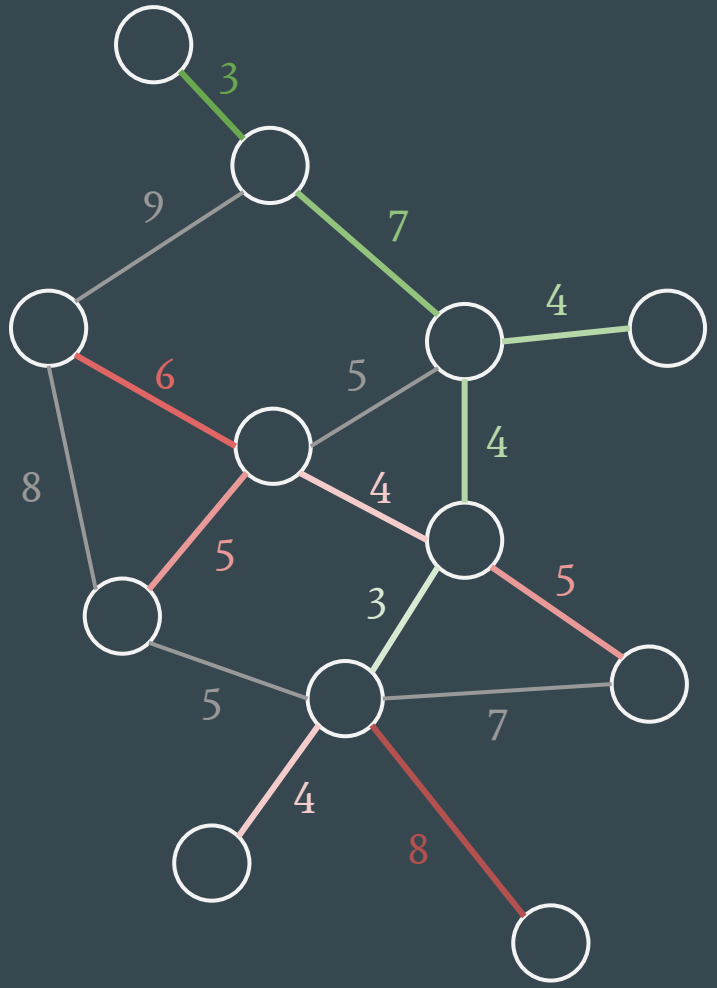
Greedy algorithm:

- put a random starting node in the tree
- while the tree doesn't have all the nodes:
  - select the node closest to the tree
  - add the corresponding edge in the tree

Implemented with a heap (remember Dijkstra last week ?)

→ Complexity : `O((|V|+|E|) log|E|)`
/!\ with a binary heap

(on the right, from green to red, order of addition in the tree)



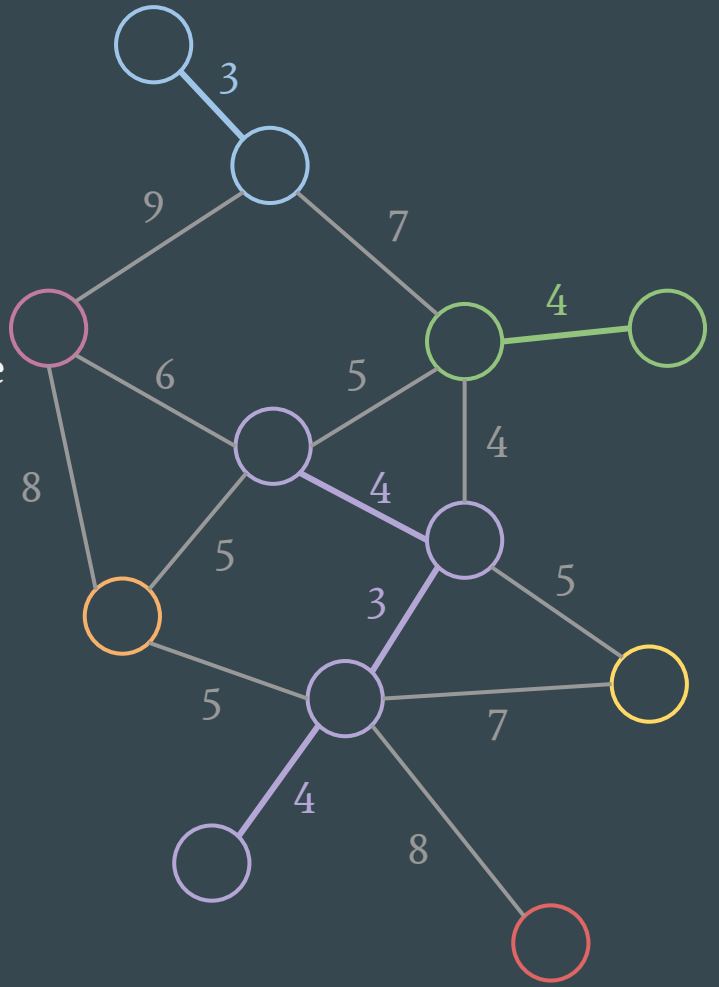
Kruskal's algorithm

Greedy algorithm:

- Create a trivial forest (all nodes are alone)
- For as long as we can, we use the smallest edge that creates a bridge between two trees

(on the right, example of a forest at some point during execution)

How to avoid making cycles ?

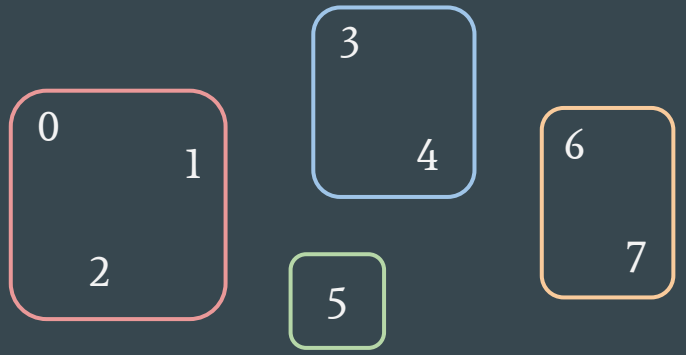


Disjoint sets (union-find d.s.)

We need a data structure to store disjoint sets with the following near-constant-time operations:

- add a new set
- merge two sets
- determine whether two elements are in the same set

Naive idea: let's use a list of sets



`[{0,1,2}, {3,4}, {5}, {6,7}]`

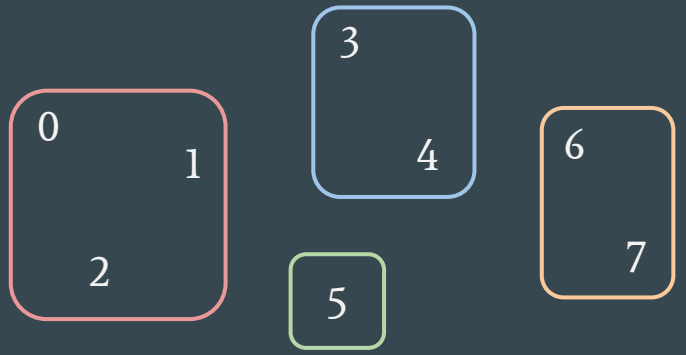
Add a new set	O(?)
Merge two sets	O(?)
Determine whether 2 elements are in the same set	O(?)

Disjoint sets (union-find d.s.)

We need a data structure to store disjoint sets with the following near-constant-time operations:

- add a new set
- merge two sets
- determine whether two elements are in the same set

Naive idea: let's use a list of sets



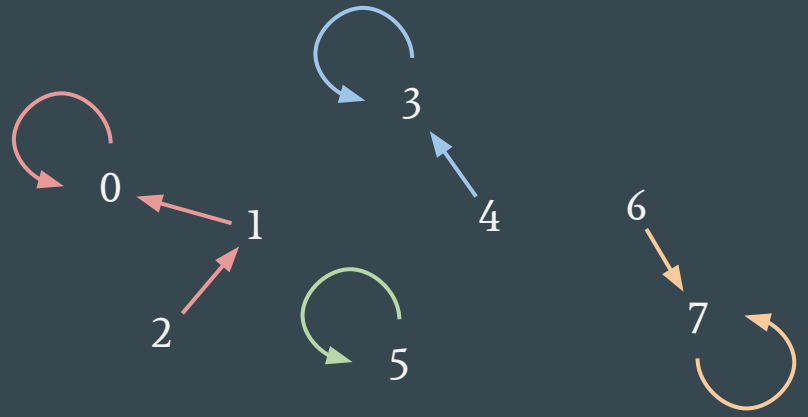
`[{0,1,2}, {3,4}, {5}, {6,7}]`

Add a new set	`O(1)`
Merge two sets	`O(m)`
Determine whether 2 elements are in the same set	`O(n)`

Disjoint sets (union-find d.s.)

Better idea: let's use a forest

- each node has a parent, the root of every tree is its own parent
- to merge two sets, point the root node of a set to any node of the other
  (you will usually use the *find* operation before, to check if the two sets are disjoint or the same)
- to determine whether 2 elements are in the same set, find the roots of their trees and compare them



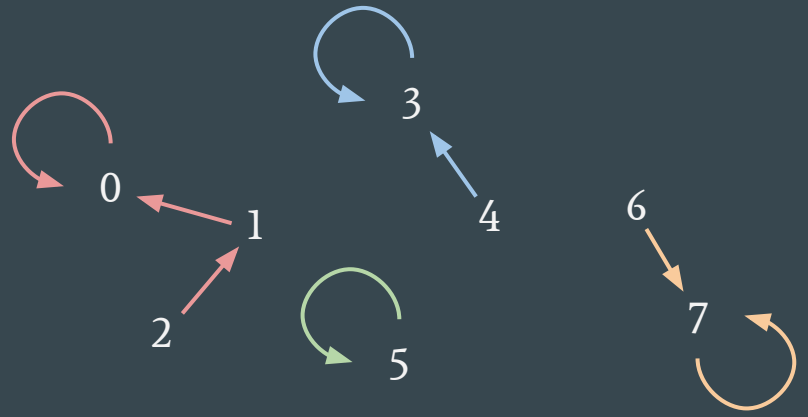

```
{0:0, 1:0, 2:1, 3:3,
 4:3, 5:5, 6:7, 7:7}
```

Add a new set	O(?)
Merge two sets	O(?)
Determine whether 2 elements are in the same set	O(?)

Disjoint sets (union-find d.s.)

Better idea: let's use a forest

- each node has a parent, the root of every tree is its own parent
- to merge two sets, point the root node of a set to any node of the other
  (you will usually use the *find* operation before, to check if the two sets are disjoint or the same)
- to determine whether 2 elements are in the same set, find the roots of their trees and compare them




```
{0:0, 1:0, 2:1, 3:3,
 4:3, 5:5, 6:7, 7:7}
```

Add a new set	`O(1)`
Merge two sets	`O(log(m))`
Determine whether 2 elements are in the same set	`O(log(m))`

Disjoint sets (union-find d.s.)

How to improve complexity?

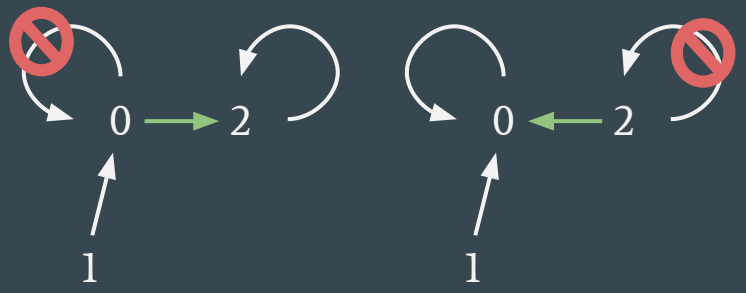
→ try to have shallow trees

→ this is achieved with ***union by rank*** (or *by size*)
and ***path compression*** (or path *halving* or *splitting*)

Add a new set	O(1)
Merge two sets	O(α(n)) *
Determine whether 2 elements are in the same set	O(α(n)) *

* α(n) < 5 for any n that can be written in this physical universe, so that's basically O(1) (see inverse Ackermann function)

path compression on the path from a node to its root during a *find* operation



arbitrary union (left) VS *union by rank* (right)

Union-find in Python

- Bad news : this structure isn't implemented by default in Python
- Good news : Louis Sugy made a library for us a few years ago ! (if lazy, just do pip install unionfind)
- Create a Union-find data structure with O(1) approximate complexities
- Use find(element) to get the root of an element
- Use union(e1, e2) to merge the sets containing e1 and e2
- Use is_same_set(e1, e2) to determine whether e1 and e2 are in the same set

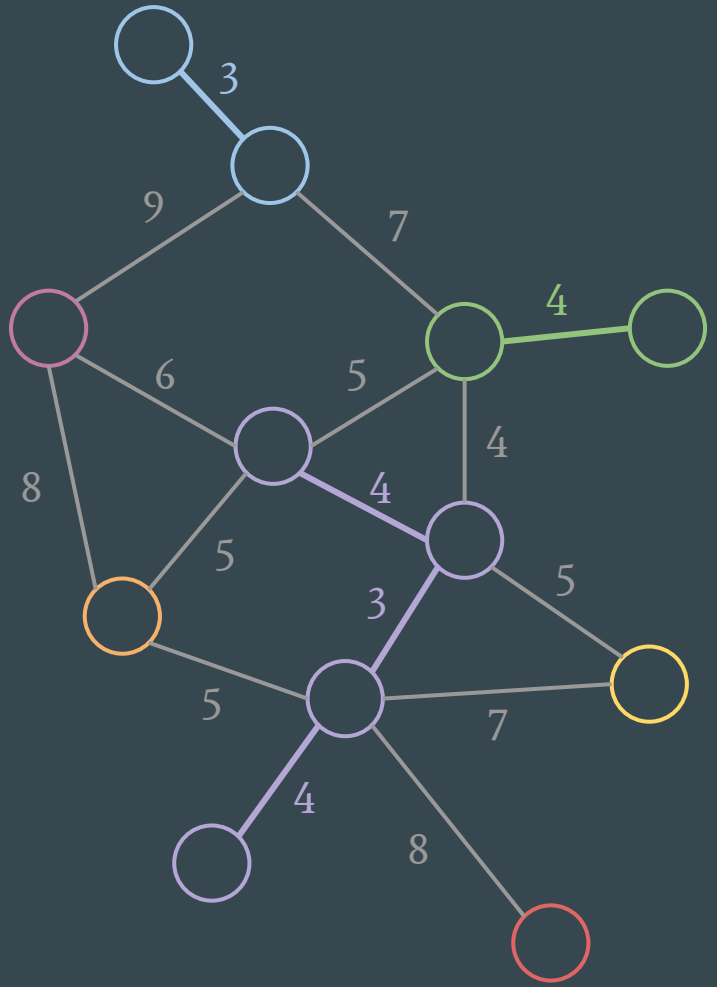
Back to Kruskal

Greedy algorithm:

- create a trivial forest (all nodes are alone)
- as long as we can, we use the smallest edge that creates a bridge between two trees

Complexity?

→ `O(|E| log |E|)` with an optimized Union-Find data structure and a heap to select the next edge



Tries

Specialized and misspelled trees

(Aka *prefix trees*)

Sample problem : how to store a phone directory

06 00 00 00 00 : John

06 00 22 45 86 : Dany

06 01 21 45 67 : Cersei

06 01 22 43 68 : Sansa

06 02 43 88 88 : Arya

07 10 11 12 13 : Night King

Sample problem : how to store a phone directory

06 00 00 00 00 : John

06 00 22 45 86 : Dany

06 01 21 45 67 : Cersei

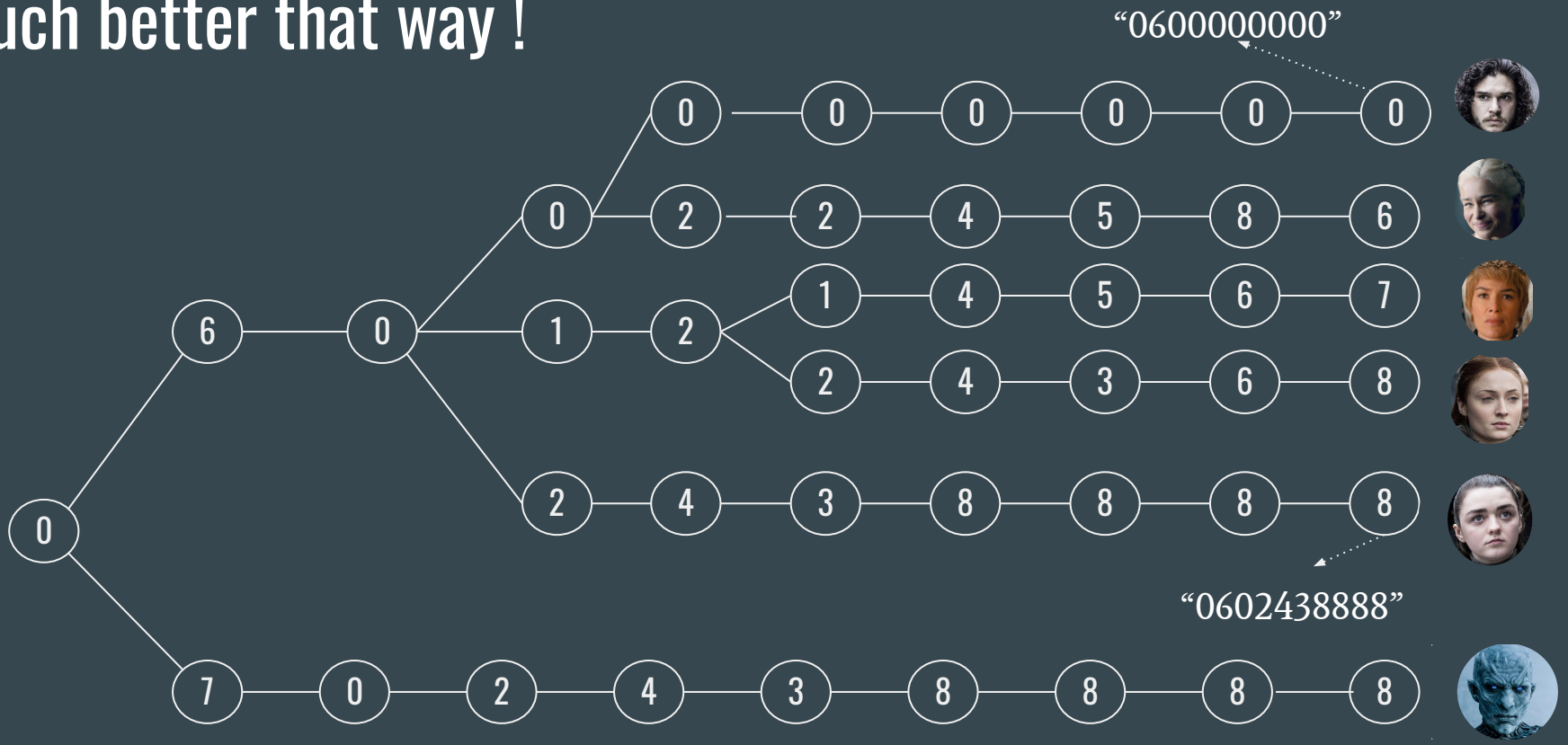
06 01 22 43 68 : Sansa

06 02 43 88 88 : Arya

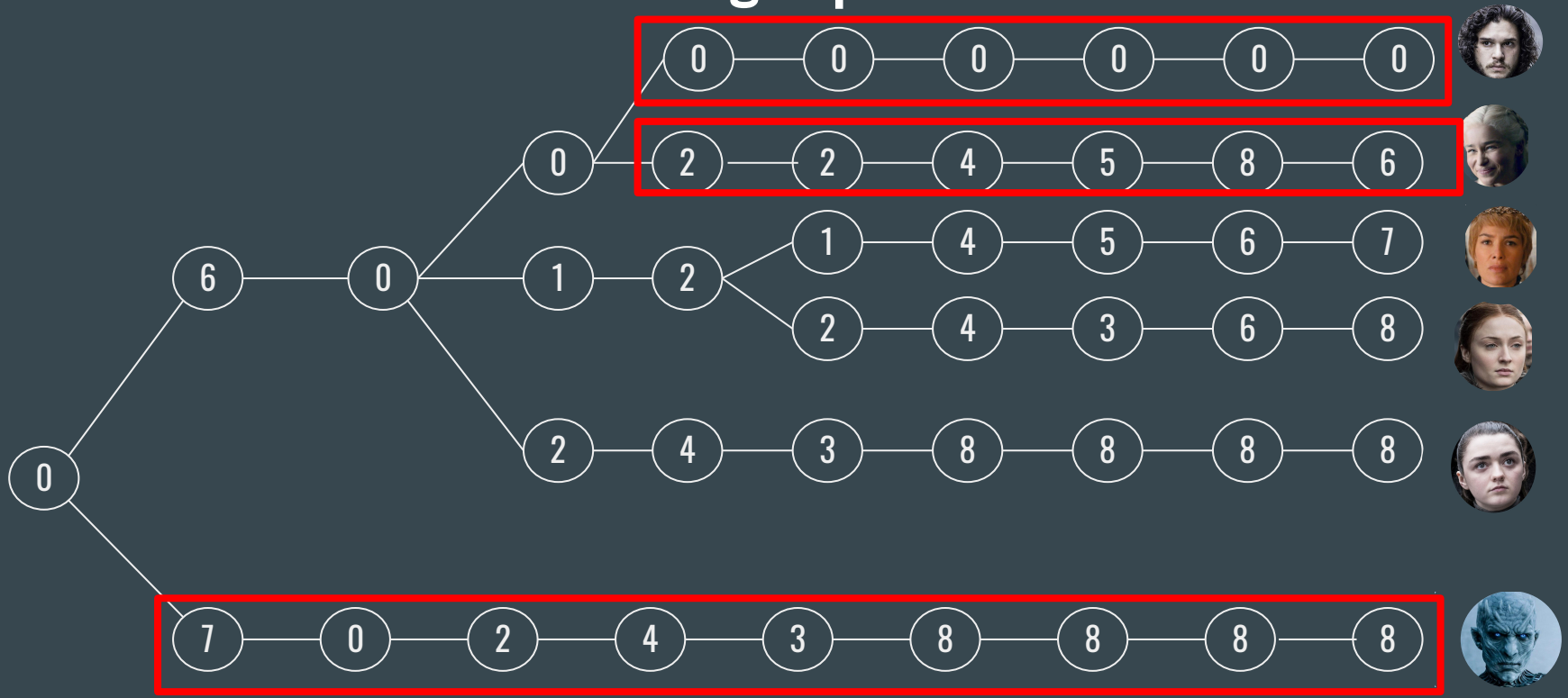
07 10 11 12 13 : Night King

Do we really need that information 5 times?

Much better that way !

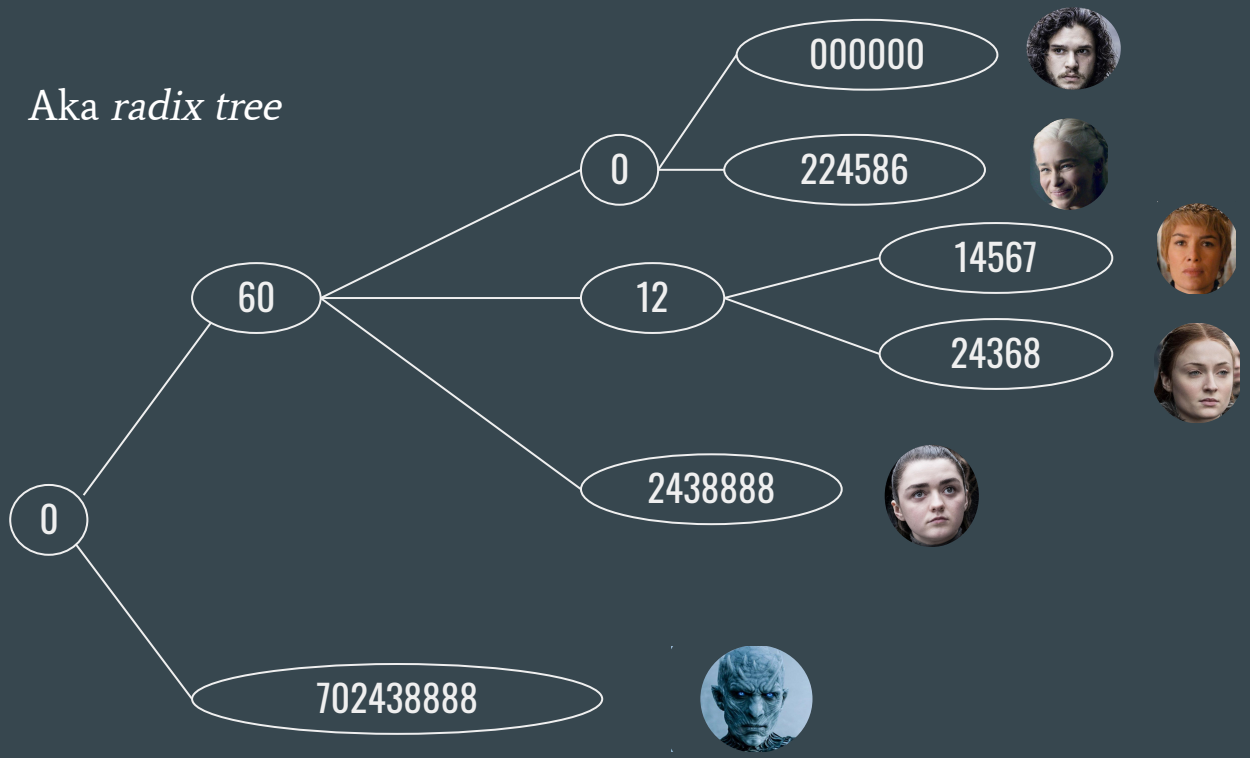


That’s a lot of nodes for a straight path



A compressed version

Aka *radix tree*



Prefix & suffix tries

- Same principle of construction, but this time the elements of the collection are prefixes/suffixes of a string

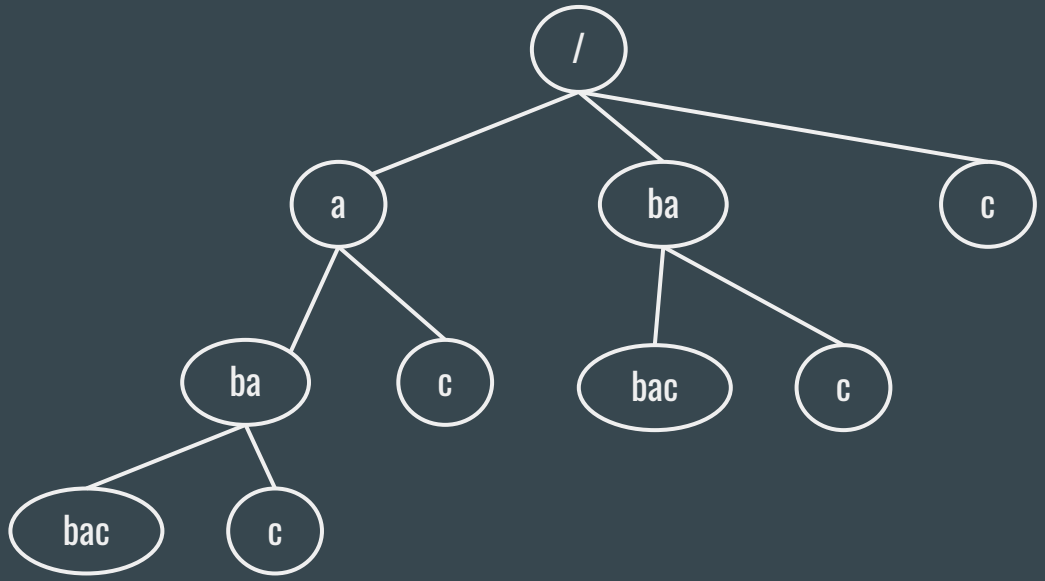
What is a suffix?

- A substring that contains the last character of a string

“INSAlgo” → { “o”, “go”, “ lgo”, “Algo”, “SAlgo”, “NSAlgo”, “INSAlgo”}

“ababac” → { “c”, “ac”, “ bac”, “abac”, “babac”, “ababac”}

Suffix tree for “ababac”



“ababac” → { “c”, “ac”, “ bac”, “abac”, “babac”, “ababac”}

How to build a suffix tree?

With n the size of the string the tree represents :

- Naive implementation runs in $O(n^2)$
  - Generate every suffix ($O(n)$), then insert them into the tree ($O(n^2)$)
- An algorithm exists to build them in $O(n)$!
  - Ukkonen's Algorithm
  - "The proof is left to the reader..."

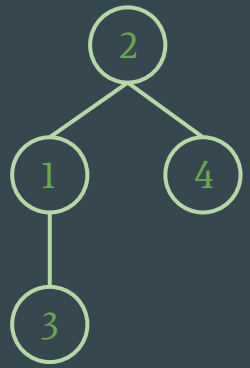
What are tries effective for?

- Data compression, auto completion
- String manipulation
  - Longest palindromic substring
  - Pattern searching
  - Longest substring common to a set of strings
  - Shortest unique substring
  - Many more !


Segment Trees
Search trees on steroids

Properties of a segment tree?

- Similar to a binary tree:
  - It's a connected & acyclic graph
  - Finding a leaf in O(log(n))
  - Linear memory usage: 4n nodes needed for n elements
- But also different!
  - Find a subarray of element that verify a property in log(n)
  - Modification of one or multiple element in a subarray in log(n)



Sum of a subarray

We are given a list of elements and asked to answer k requests for the sum of a given subarray

→ “Moving sum” array:

We create a new array where each element is the sum of all the element before it.

1	5	7	6	7	9	3

0	1	6	13	19	26	35	38

Sum of a subarray

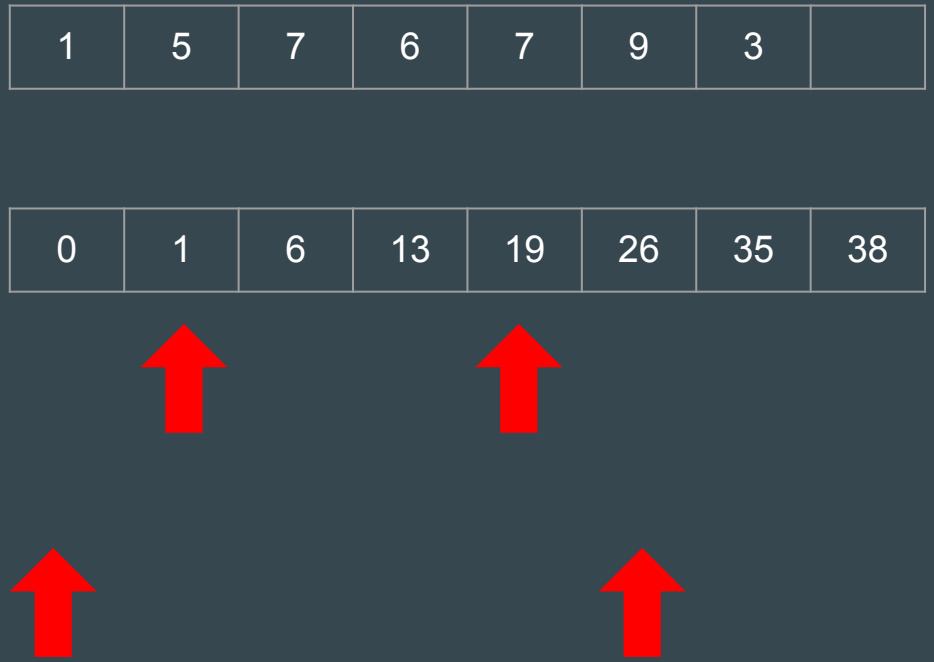
We are given a list of elements and asked to answer k requests for the sum of a given subarray

→ “Moving sum” array:

We create a new array where each element is the sum of all the element before it.

Sum of the subarray [1:3]
→ 19-1=18

Sum of the subarray [0:5]
→ 26-0=26



Sum of a subarray

We are given a list of elements and asked to answer k requests for the sum of a given subarray

But what if we also need to change a value in the array?

→ We have to recompute everything!

1	10	7	6	7	9	3	

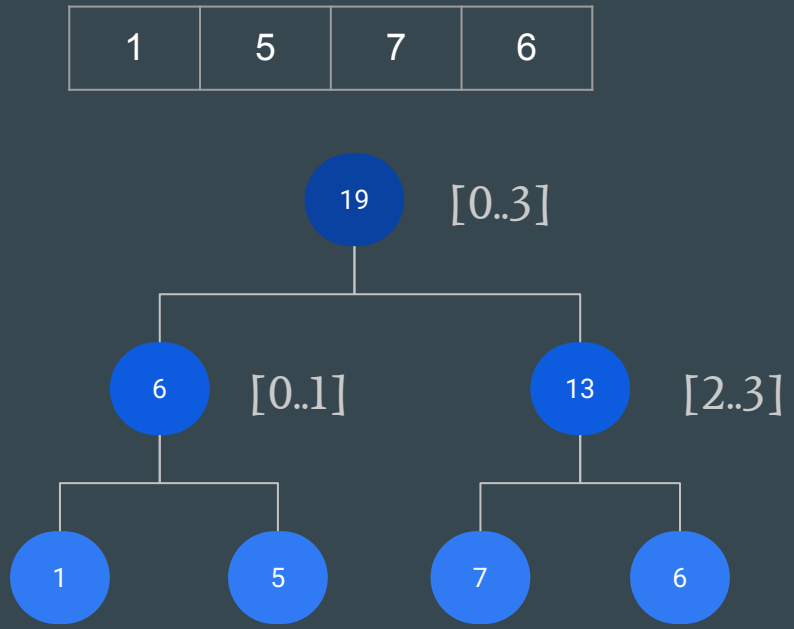
0	1	11	18	24	31	40	43

Sum of a subarray, with segment trees

Leaves are the array, in the same order. Each parent is the sum of his two children.

How to compute the sum in the subarray?

We are given a list of elements and asked to answer k requests to either update the array or to compute the sum of a subarray



Sum of a subarray, with segment trees

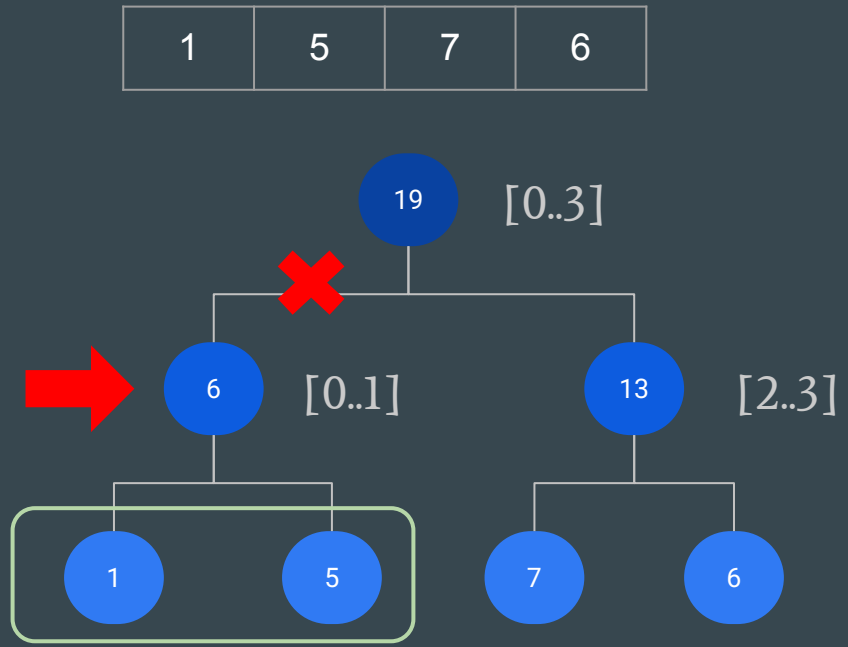
Leaves are the array, in the same order. Each parent is the sum of his two children.

How to compute the sum in the subarray?

Sum of the subarray [0:1]

→ 6

We are given a list of elements and asked to answer k requests to either update the array or to compute the sum of a subarray



Sum of a subarray, with segment trees

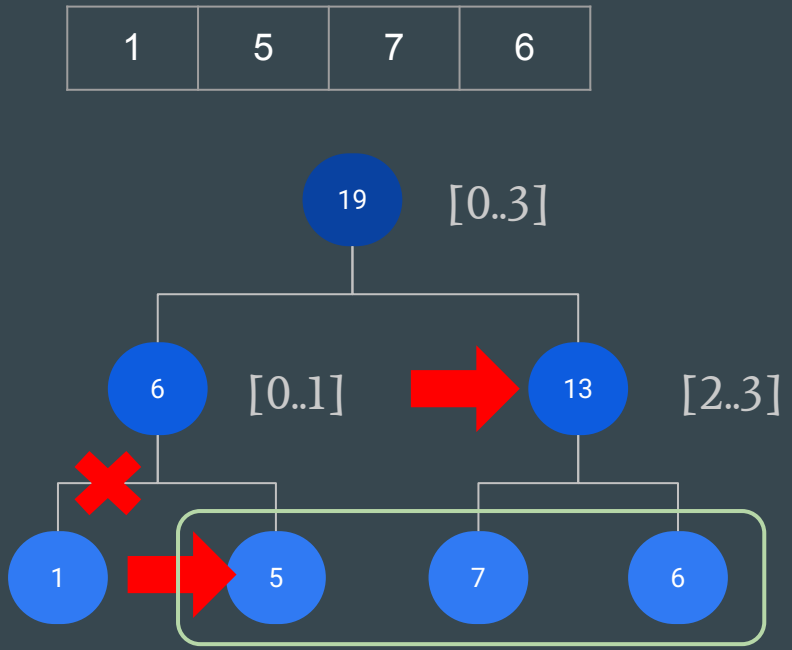
Leaves are the array, in the same order. Each parent is the sum of his two children.

How to compute the sum in the subarray?

Sum of the subarray [1:3]

→ 5+13=18

We are given a list of elements and asked to answer k requests to either update the array or to compute the sum of a subarray



Sum of a subarray, with segment trees

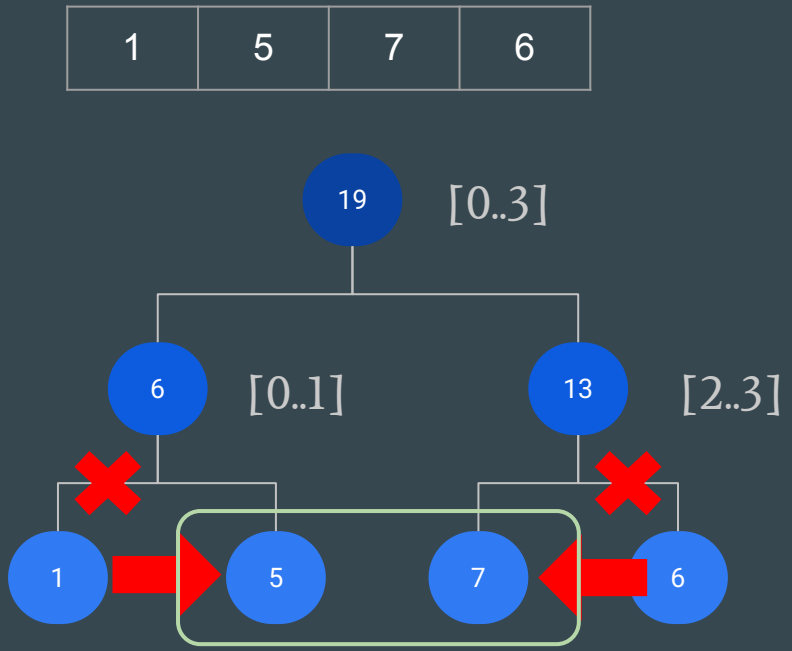
Leaves are the array, in the same order. Each parent is the sum of his two children.

How to compute the sum in the subarray?

Sum of the subarray [1:2]

→ 5+7=12

We are given a list of elements and asked to answer k requests to either update the array or to compute the sum of a subarray



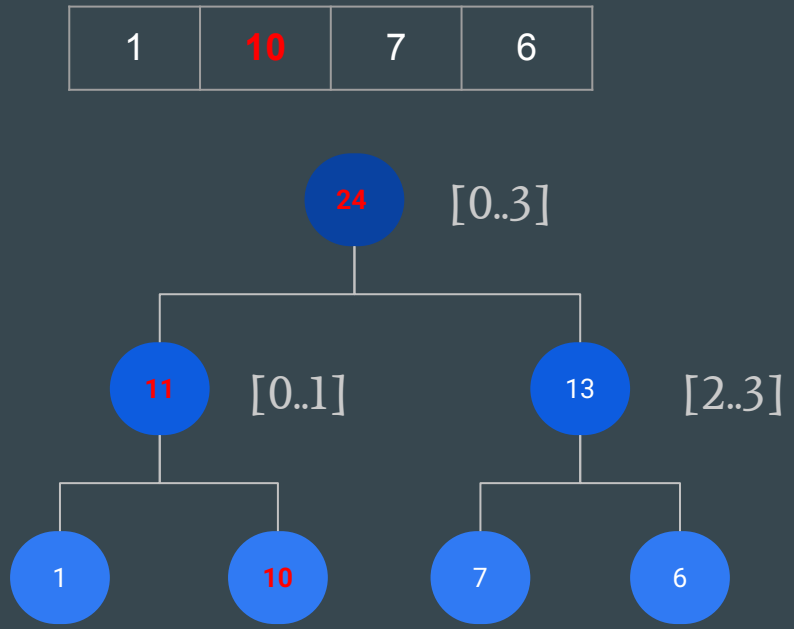
Sum of a subarray, with segment trees

And what if we need to update a value?

→ Update the leaf and all of its parents

O(log(n)) operations, this is much better than the previous O(n)

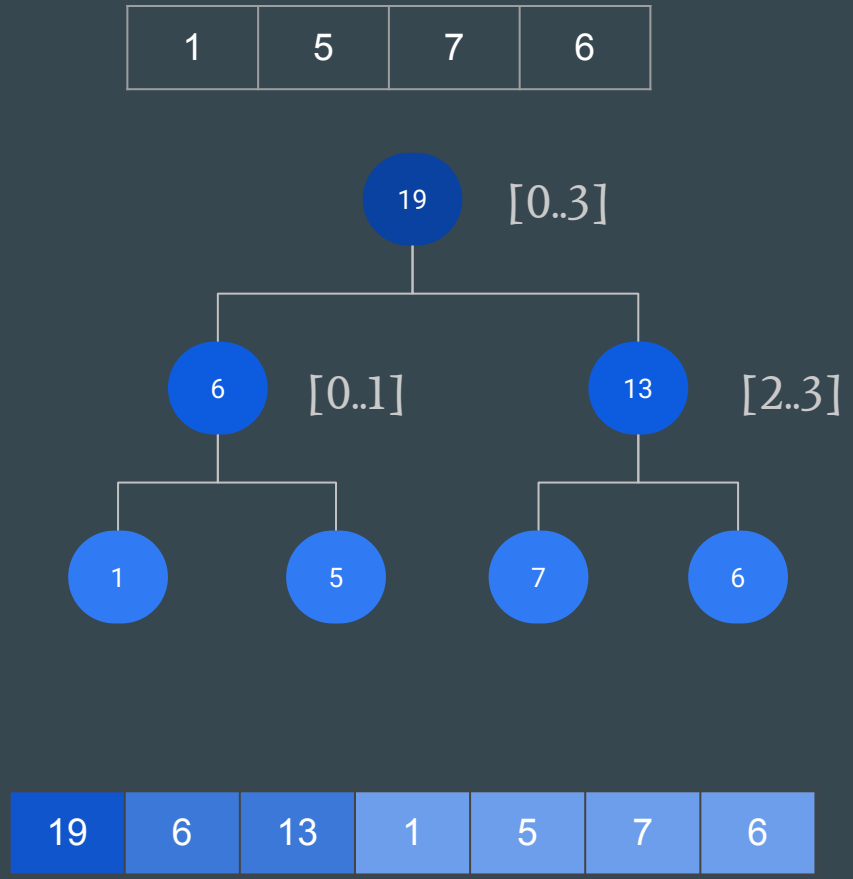
We are given a list of elements and asked to answer k requests to either update the array or to compute the sum of a subarray



Implementation of a segment tree

Similar to most binary trees.

For a node at the index i, its children are stored at the indexes `2i` and `2i+1`.



Implementation of a segment tree

Can be generalized to higher dimensions!

With a 2D matrix, we first construct the segment tree on each row (instead of each cell/value)

Then, we construct a segment tree for each node of the previous tree, on the values of each row

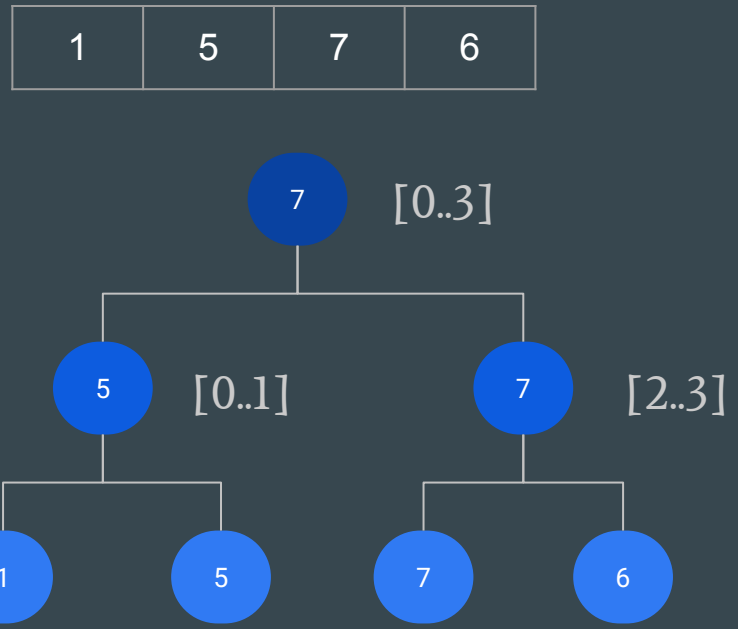
1	5	3	7
13	9	6	7
2	12	4	3
3	6	3	2

86	51	35	19	32	16	19
51	28	23	14	14	9	14
35	23	12	5	18	7	5
16	6	10	1	5	3	7
35	22	13	13	9	6	7
21	14	7	2	12	4	3
14	9	5	3	6	3	2

Generalization

The heuristic can be adapted to solve a wide range of problems.

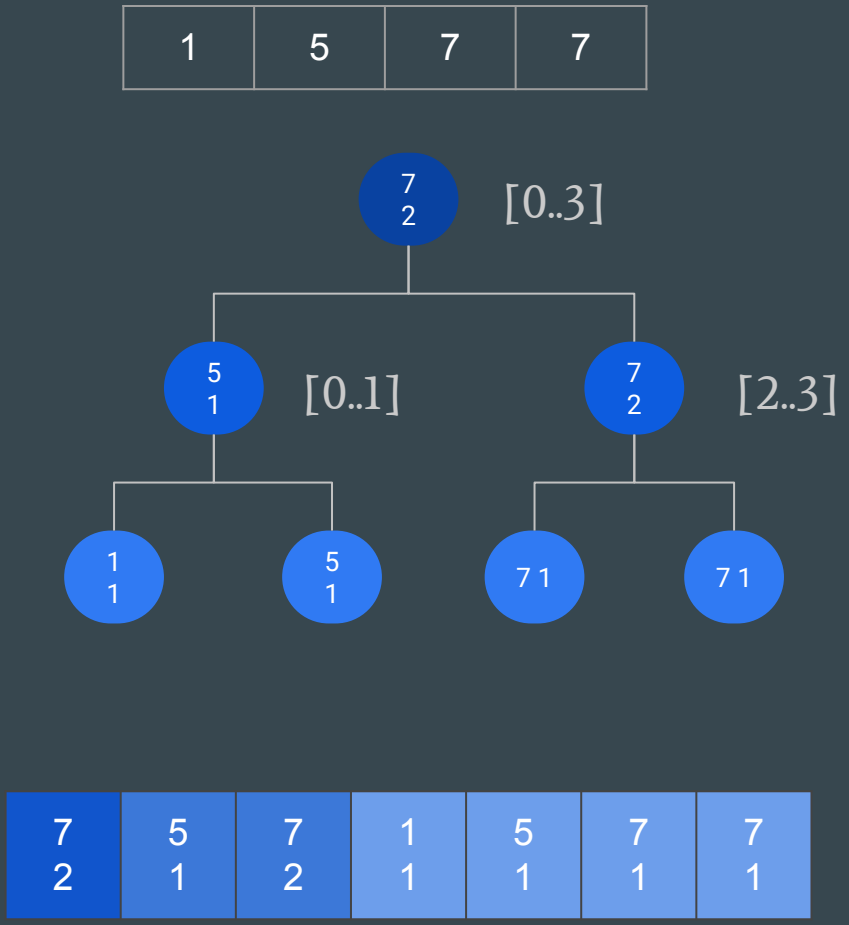
→ Search of the local maximum



Generalization

The heuristic can be adapted to solve a wide range of problems.

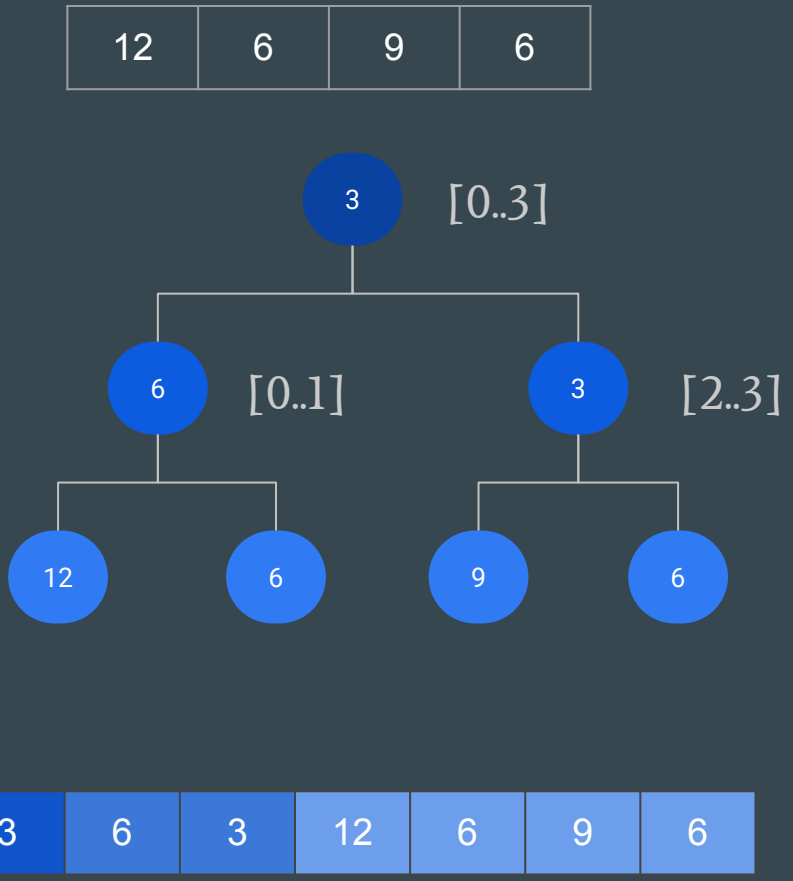
→ Search of the local maximum, and how many times it appears



Generalization

The heuristic can be adapted to solve a wide range of problems.

→ Computing the GCD (or LCM) of a subarray



Implementation

You can implement most
segment trees using 4 functions:

merge()	build()	update()	query()
Compute a node value from its children's value	Generate the segment tree from a dataset	Updates a value from the dataset and updates the necessary nodes	Compute a property on a given subarray of the dataset

Implementation: merge

Defines how a node value depends on its children.

It is usually called by all other three functions.

```
def merge_func(self, node_left, node_right):
        return node_left + node_right
```

Implementation: build

Creates the tree from a dataset (here from self.data)

It is usually called only once and runs in O(n) if the merge function runs in O(1), because there are n internal nodes in the tree.

```
self.segtree = [0] * (4 * len(data))

def build(self, node, start, end):
    if start == end:
        self.segtree[node] = self.data[start]
    else:
        mid = (start + end) // 2
        self.build(2 * node + 1, start, mid)
        self.build(2 * node + 2, mid + 1, end)
        self.segtree[node] = self.merge(
            self.segtree[2 * node + 1],
            self.segtree[2 * node + 2]
        )
```

Implementation: build

Note:

For a node of index i, its children's indexes are **2i** and **2i+1** (2i+1 and 2i+2 if our array is 0-indexed).

This is **NOT** the most optimal indexing method memory-wise, as a segment tree only requires 2n-1 vertices, but it is easier to implement.

```
self.segtree = [0] * (4 * len(data))

def build(self, node, start, end):
    if start == end:
        self.segtree[node] = self.data[start]
    else:
        mid = (start + end) // 2
        self.build(2 * node + 1, start, mid)
        self.build(2 * node + 2, mid + 1, end)
        self.segtree[node] = self.merge(
            self.segtree[2 * node + 1],
            self.segtree[2 * node + 2]
        )
```

Implementation: update

Updates a value from the dataset and updates the affected internal nodes.

This function could be optimized for updating a subarray of the dataset, by generating a smaller segment tree and inserting it.

```
def update(self, node, start, end, idx, value):
    if start == end:
        self.data[idx] = value
        self.segtree[node] = value
    else:
        mid = (start + end) // 2
        if idx <= mid:
            self.update(2*node+1, start, mid, idx, value)
        else:
            self.update(2*node+2, mid+1, end, idx, value)

        self.segtree[node] = self.merge(
            self.segtree[2*node+1],
            self.segtree[2*node+2]
        )
```

Implementation: query

Efficiently compute a property on a given subarray of the dataset, using the segment tree.

This function's implementation might change a lot depending on the property.

```
def query(self, node, start, end, L, R):
    if R < start or end < L:
        return 0

    if L == start and end == R:
        return self.segtree[node]

    mid = (start + end) // 2
    left_sum = self.query(
        2*node+1,
        start, mid,
        L, min(R, mid)
    )
    right_sum = self.query(
        2*node+2,
        mid+1, end,
        max(L, mid+1), R
    )

    return self.merge(left_sum, right_sum)
```

Credits:

Binary Search Trees	Graph Theory	Tries	Segment Trees
Slides: Lecorché Adriaan	Slides: Louis Sugy Edited by: Goll Sebastien, Lecorché Adriaan	Slides: Arthur Tondereau Edited by: Emma Neiss	Slides: William Michaud